

PROJET UVSQolor

13 MAI 2026

Olivia Gomis	YASMINE ABADI
--------------	---------------

Sommaire

- 01 Introduction - Structure générale | Yasmine
- 02 L'interface graphique | Olivia
- 03 Filtres de couleurs - TD | Yasmine
- 04 Fonctions Charger_fichier, Rafraichir, Supprimer_fichier, Valider | Olivia
- 05 Fonctions Sauvegarder_etat, Annuler_filtre, Retablir_filtre | Yasmine
- 06 Filtres Miroir, Fusionner, Inverse | Olivia
- 07 Filtres de Convolusion, Niveaux de gris, Warhol | Yasmine
- 08 Conclusion, démonstartion | Olivia

- Importation des différentes bibliothèques
- Définition des variables en global
- Création de l'interface graphique : la fenêtre principale, les Canvas, les fonctions d'affichage avec leurs boutons, le menu déroulant, les zones de texte
- Différentes fonctions de filtres

PRÉSENTATION GÉNÉRALE DU SUJET

LES BIBLIOTHEQUES

```
import numpy as np
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from scipy.signal import convolve2d
```

FENETRE PRINCIPALE

```
#création de la fenetre et du canva servant d'interface
fenetre=tk.Tk()
fenetre.title("Application de filtres | PROJET")
fenetre['bg']="#FDE7EB"
fenetre.geometry("700x550")
tableau=tk.Canvas(fenetre,width=600,height=300,bg="white")
tableau.pack()
```

FONCTIONS DE FILTRE

```
def appliquer_inverse():
    global image_pil
    if image_pil:
        image_numpy = np.array(image_pil)
        image_numpy = 255-image_numpy
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))
        sauvegarder_etat()
        rafraichir()
```

TEXTE

```
txt_filtres=tk.Label(fenetre,text="🏠-----Filtres-----🏠",bg="#FDE7EB")
txt_filtres.pack()
```

VARIABLES GLOBALES

```
#setting des variables de départ
image_pil = None
image_tk = None
image_originale = None
historique=[]
indice_historique = -1
```

FONCTIONS DE MANIPULATION DES IMAGES

```
def annuler_filtre():
    global indice_historique,historique,image_pil
    if indice_historique>0:
        indice_historique-=1
        image_numpy = historique[indice_historique].copy()
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))
        rafraichir()

def retablir_filtre():
    global indice_historique,historique,image_pil
    if indice_historique<len(historique)-1:
        indice_historique+=1
        image_numpy = historique[indice_historique].copy()
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))
        rafraichir()
```

L'INTERFACE GRAPHIQUE

FILTRES - TD | YASMINE

SEPIA

```
def appliquer_sepia():
    global image_pil
    image_numpy = np.array(image_pil).astype(float)
    r = image_numpy[:, :, [0]]
    v = image_numpy[:, :, [1]]
    b = image_numpy[:, :, [2]]
    r_prime = r + 0.5 * v + 0.1 * b
    v_prime = 0.5 * r + 0.8 * v + 0.1 * b
    b_prime = 0.2 * r + 0.3 * v + 0.5 * b
    r_prime = np.clip(r_prime, 0, 255)
    v_prime = np.clip(v_prime, 0, 255)
    b_prime = np.clip(b_prime, 0, 255)
    image_numpy[:, :, [0]] = r_prime
    image_numpy[:, :, [1]] = v_prime
    image_numpy[:, :, [2]] = b_prime
    image_numpy = image_numpy.astype(np.uint8)
    image_pil = Image.fromarray(image_numpy)
    sauvegarder_etat()
    rafraichir()
```

LUMINOSITÉ

```
def appliquer_lumi():
    global image_pil
    if image_pil:
        image_numpy = np.array(image_pil).astype(float)
        r = image_numpy[:, :, 0] * 1.2
        v = image_numpy[:, :, 1] * 1.2
        b = image_numpy[:, :, 2] * 1.2
        r = np.clip(r, 0, 255)
        v = np.clip(v, 0, 255)
        b = np.clip(b, 0, 255)
        image_numpy = np.dstack((r, v, b))
        image_numpy = image_numpy.astype(np.uint8)
        image_pil = Image.fromarray(image_numpy)
        sauvegarder_etat()
        rafraichir()
```

CONTRASTE

```
def appliquer_contraste():
    global image_pil
    if image_pil:
        matrice_pixels = np.array(image_pil).astype(float)
        valeur_contraste = 0.15
        r = matrice_pixels[:, :, [0]]
        v = matrice_pixels[:, :, [1]]
        b = matrice_pixels[:, :, [2]]
        r_prime = (r - 128) * valeur_contraste + 128
        v_prime = (v - 128) * valeur_contraste + 128
        b_prime = (b - 128) * valeur_contraste + 128
        r_prime = np.clip(r_prime, 0, 255)
        v_prime = np.clip(v_prime, 0, 255)
        b_prime = np.clip(b_prime, 0, 255)
        matrice_pixels[:, :, [0]] = r_prime
        matrice_pixels[:, :, [1]] = v_prime
        matrice_pixels[:, :, [2]] = b_prime
        image_pil = Image.fromarray(matrice_pixels.astype(np.uint8))
        rafraichir()
```


Fonctions Charger_fichier, Rafraichir, Supprimer_fichier,
Valider | Olivia

RAFRAICHIR

```
def rafraichir():  
    global image_tk  
    if image_pil:  
        image_tk = ImageTk.PhotoImage(image_pil)  
        tableau.delete("all")  
        tableau.config(width=600, height=300)  
        tableau.create_image(0,0,anchor=tk.NW,image=image_tk)
```

CHARGER_FICHER

```
def charger_fichier():  
    global image_pil,image_originale,historique,indice_historique  
    chemin_image=filedialog.askopenfilename(title="Image à charger")  
    if chemin_image:  
        image_pil = Image.open(chemin_image).convert("RGB")  
        image_pil = image_pil.resize((600,300))  
        image_originale = image_pil.copy()  
        historique = []  
        indice_historique = -1  
        sauvegarder_etat()  
        rafraichir()
```

BOUTON FICHER

```
bouton_fichier=tk.Button(cadre_boutons,text="Charger une image",bg="lightpink",font=("Times",12),command=charger_fichier)  
bouton_fichier.pack(side="left")
```

#SUPPRIMER_FICHER

```
def supprimer_fichier():  
    global image_pil,image_originale,image_tk  
    tableau.delete("all")  
    image_pil = None  
    image_originale = None  
    image_tk = None
```

VALIDER

```
def valider():  
    filtre=menu.get()  
    if filtre=="SEPIA":  
        appliquer_sepia()  
    elif filtre=="LUMINOSITE":  
        appliquer_lumi()  
    elif filtre=="CONTRASTE":  
        appliquer_contraste()  
    elif filtre=="FLOU":  
        appliquer_flou()  
    elif filtre=="FLOU GAUSSIEN":  
        appliquer_gauss()  
    elif filtre=="NETTETE":  
        appliquer_net()  
    elif filtre=="FUSIONNER":  
        fusionner()  
    elif filtre=="MIROIR":  
        appliquer_miroir()  
    elif filtre=="INVERSE":  
        appliquer_inverse()  
    elif filtre=="NOIR ET BLANC":  
        appliquer_noir_et_blanc()  
    elif filtre=="ANDY WARHOL":  
        appliquer_warhol()
```

SAUVEGARDER_ETAT

```
def sauvegarder_etat():
    global historique, indice_historique, image_pil
    if image_pil is None:
        return
    image_numpy = np.array(image_pil)
    if len(historique) - 1 > indice_historique:
        historique = historique[:indice_historique+1]
    historique.append(image_numpy.copy())
    indice_historique = len(historique)-1
```

#ANNULER_FILTRE

```
def annuler_filtre():
    global indice_historique, historique, image_pil
    if indice_historique > 0:
        indice_historique -= 1
        image_numpy = historique[indice_historique].copy()
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))
        rafraichir()
```

#RETABLIR_FILTRE

```
def retablir_filtre():
    global indice_historique, historique, image_pil
    if indice_historique < len(historique)-1:
        indice_historique += 1
        image_numpy = historique[indice_historique].copy()
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))
        rafraichir()
```

BOUTONS ASSOCIES

```
barre_menu = tk.Menu(cadre_menu, bg="#F4EDE0", tearoff=0)
menu_fichier = tk.Menu(barre_menu, tearoff=0)
menu_fichier.add_command(label="Annuler le filtre", command=annuler_filtre)
menu_fichier.add_command(label="Rétablir le filtre", command=retablir_filtre)
barre_menu.add_cascade(label="Fichier", menu=menu_fichier)
fenetre.config(menu=barre_menu)
```


Filtres Miroir, Fusionner, Inverse |Olivia

#MIROIR

```
def appliquer_miroir():  
    global image_pil  
    if image_pil:  
        image_numpy = np.array(image_pil)  
        image_numpy = image_numpy[:,::-1]  
        image_pil = Image.fromarray(image_numpy)  
        sauvegarder_etat()  
        rafraichir()
```

#FUSIONNER

```
def fusionner():  
    global image_pil  
    if image_pil:  
        chemin2 = filedialog.askopenfilename(title="Image à fusionner")  
        if chemin2:  
            img2 = Image.open(chemin2).convert("RGB").resize((600, 300))  
            matrice1 = np.array(image_pil).astype(float)  
            matrice2 = np.array(img2).astype(float)  
            fusion = (matrice1 + matrice2) / 2  
            image_pil = Image.fromarray(np.uint8(fusion))  
            sauvegarder_etat()  
            rafraichir()
```

#INVERSE

```
def appliquer_inverse():  
    global image_pil  
    if image_pil:  
        image_numpy = np.array(image_pil)  
        image_numpy = 255-image_numpy  
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))  
        sauvegarder_etat()  
        rafraichir()
```


Filtres de Convolusion, Niveaux de gris, Warhol | Yasmine

FLOU GAUSSIEN

```
def appliquer_gauss():
    global image_pil
    if image_pil:
        matrice= np.array(image_pil).astype(float)
        kernel=noyau/noyau.sum()
        new_matrice = np.zeros_like(matrice,dtype=float)
        for i in range (3):
            new_matrice[:, :,i] = convolve2d(matrice[:, :,i],kernel, mode="same",boundary="symm")
        matrice_pixels=np.clip(new_matrice, 0, 255).astype(np.uint8)
        image_pil = Image.fromarray(matrice_pixels.astype(np.uint8))
        sauvegarder_etat()
        rafraichir()
```

```
noyau_fort= np.array([[0,-1,0],
                      [-1,5,-1],
                      [0,-1,0]])
```

```
noyau = np.array([[1, 4, 6, 4, 1],
                  [4, 16, 24, 16, 4],
                  [6, 24, 36, 24, 6],
                  [4, 16, 24, 16, 4],
                  [1, 4, 6, 4, 1]]) / 256
```

```
noyau2 = np.array([[1,1,1,1,1],
                  [1,1,1,1,1],
                  [1,1,1,1,1],
                  [1,1,1,1,1],
                  [1,1,1,1,1]]) / 256
```

#NETTETE

```
def appliquer_net():
    global image_pil
    if image_pil:
        float_matrice=np.array(image_pil).astype(float)
        new_matrice = np.zeros_like(float_matrice)
        for i in range (3): #convolution sur les 3 canaux; boundary = symm => évite bords noirs
            new_matrice[:, :,i] = convolve2d(float_matrice[:, :,i],noyau_fort, mode="same",boundary="symm")
        matrice_pixels=np.clip(new_matrice, 0, 255).astype(np.uint8)
        image_pil = Image.fromarray(matrice_pixels.astype(np.uint8))
        sauvegarder_etat()
        rafraichir()
```

#FLOU

```
def appliquer_flou():
    global image_pil
    if image_pil:
        matrice= np.array(image_pil).astype(float)
        kernel=noyau2/noyau2.sum()
        new_matrice = np.zeros_like(matrice,dtype=float)
        for i in range (3):
            new_matrice[:, :,i] = convolve2d(matrice[:, :,i],kernel, mode="same",boundary="symm")
        matrice_pixels=np.clip(new_matrice, 0, 255).astype(np.uint8)
        image_pil = Image.fromarray(matrice_pixels.astype(np.uint8))
        sauvegarder_etat()
        rafraichir()
```

#NIVEAUX DE GRIS

```
def appliquer_noir_et_blanc():  
    global image_pil  
    if image_pil:  
        image_numpy = np.array(image_pil).astype(float)  
        r = image_numpy[:, :, [0]]  
        v = image_numpy[:, :, [1]]  
        b = image_numpy[:, :, [2]]  
        r_prime = 0.299*r  
        v_prime = 0.587*v  
        b_prime = 0.114*b  
        r_prime = np.clip(r_prime, 0, 255)  
        v_prime = np.clip(v_prime, 0, 255)  
        b_prime = np.clip(b_prime, 0, 255)  
        Y = r_prime + v_prime + b_prime  
        image_numpy[:, :, [0, 1, 2]] = Y  
        image_pil = Image.fromarray(image_numpy.astype(np.uint8))  
        sauvegarder_etat()  
        rafraichir()
```

Filtres de Convolusion, Niveaux de gris, Warhol | Yasmine

#WARHOL

```
def appliquer_warhol():
    global image_pil
    if image_pil:
        matrice_pixels=np.array(image_pil).astype(float)
        h, w, c = matrice_pixels.shape # recup les 3 canaux
        image_1 = matrice_pixels.copy()
        image_2 = matrice_pixels.copy()
        image_3 = matrice_pixels.copy()
        image_4 = matrice_pixels.copy()

        image_1[:, :, 0] *=1.2
        image_1[:, :, 1] *=0.9
        image_1[:, :, 2] *=0.4

        image_2[:, :, 0] *=0.2
        image_2[:, :, 1] *=1.1
        image_2[:, :, 2] *=0.4

        image_3[:, :, 0] *=0.74
        image_3[:, :, 1] *=0.543
        image_3[:, :, 2] *=1.34
```

```
image_4[:, :, 0] *=1.13
image_4[:, :, 1] *=0.129
image_4[:, :, 2] *=1.13

image_1 = np.clip(image_1, 0, 255).astype(np.uint8)
image_2 = np.clip(image_2, 0, 255).astype(np.uint8)
image_3 = np.clip(image_3, 0, 255).astype(np.uint8)
image_4 = np.clip(image_4, 0, 255).astype(np.uint8)

#matrice finale qui fait 2 fois la taille initiale pour faire rentrer les 4 images
matrice_finale=np.zeros((h*2,w*2,c), dtype=np.uint8)

matrice_finale[0:h, 0:w]=image_1
matrice_finale[h:h*2, 0:w]=image_2
matrice_finale[0:h, w:w*2]=image_3
matrice_finale[h:h*2, w:w*2]=image_4

matrice_pixels = matrice_finale

image_pil=Image.fromarray(matrice_pixels)
image_pil = image_pil.resize((600, 300))
sauvegarder_etat()
rafraichir()
```

CONCLUSION

TEMPS D'ÉCHANGES

*Avez-vous
des questions ?*

Merci !

13 MAI 2026

Olivia Gomis	YASMINE ABADI
--------------	---------------